



Digitalna platforma za upravljanje teretanom

Objektni model

Amelija Trifunović, Bogdana Marić, Milica Bobar, Jelena Matijaš, Jana Ivanković

Verzija: 1.0 | Datum: AVGUST 2026.

SADRŽAJ

1. Uvod	3
2. Domenski dijagram klasa	4
3. Frontend	5
3.1. Inicijalizacija sistema	5
3.2. Pregled naloga.....	7
3.3. Pregled planova treninga	9
3.4 Pregled prijavljenih problema.....	10
4. Backend.....	14
4.1. Raspored grupnih treninga	14
4.2. Prijava problema.....	16
4.3. Administratorski i korisnički nalog	18
4.4. Upravljanje terminima.....	20
4.5 Upravljanje planovima treninga.....	22

1. Uvod

U ovom dokumentu je predstavljen objektni model za softverski sistem pumpd upotrebom UML dijagrama klasa. Dokument je podijeljen u tri cjeline: domenski dijagram klasa, klijentski (frontend) dio i pozadinski (backend) dio.

Klijentski (frontend) dio prikazuje strukturu i komunikaciju na strani korisnika (mobilna aplikacija razvijena u React Native tehnologiji) te slanje podataka putem mreže prema aplikativnom serveru. Aplikacija je realizovana kao varijacija MVVM (Model-View-ViewModel) arhitekturnog obrasca sa sljedećim razlikama u organizaciji:

- **ViewModel:** U aplikaciji je predstavljen kroz ekrane i reupotrebljive komponente. Ove komponente obavljaju poslovnu logiku prikaza i upravljaju interakcijom sa korisnikom. Na primjer, komponente koriste `useEffect`, `useState` i `useNavigation` za upravljanje stanjima, pozivanje funkcija i navođenje odgovarajućih akcija, kao što je navigacija između ekrana. Sve funkcije koje se tiču interakcije s korisnikom i manipulacije podacima predstavljaju ViewModel sloj.
- **View:** View u ovoj aplikaciji je striktno odvojen od poslovne logike i obuhvata samo stilizovanje i prikazivanje korisničkog interfejsa. To uključuje zbirke stilova (Styles klase) koje definišu izgled elemenata na ekranu. Takođe, JSX je dio View sloja, jer se koristi za deklarativno strukturisanje i organizovanje korisničkog interfejsa u aplikaciji.

Na ovaj način, ViewModel (ekrani i komponente) komunicira sa korisnikom i obrađuje logiku, dok View (Styles i JSX) obavlja isključivo prikazivanje podataka i stilizovanje interfejsa. Ovaj pristup omogućava jasnu podjelu odgovornosti (separation of concerns). Ekрани i komponente direktno obrađuju podatke koje dobijaju preko klijentskog API sloja (api / HttpService klase), koji služi za prosljeđivanje REST HTTP zahtjeva prema backend serveru aplikacije.

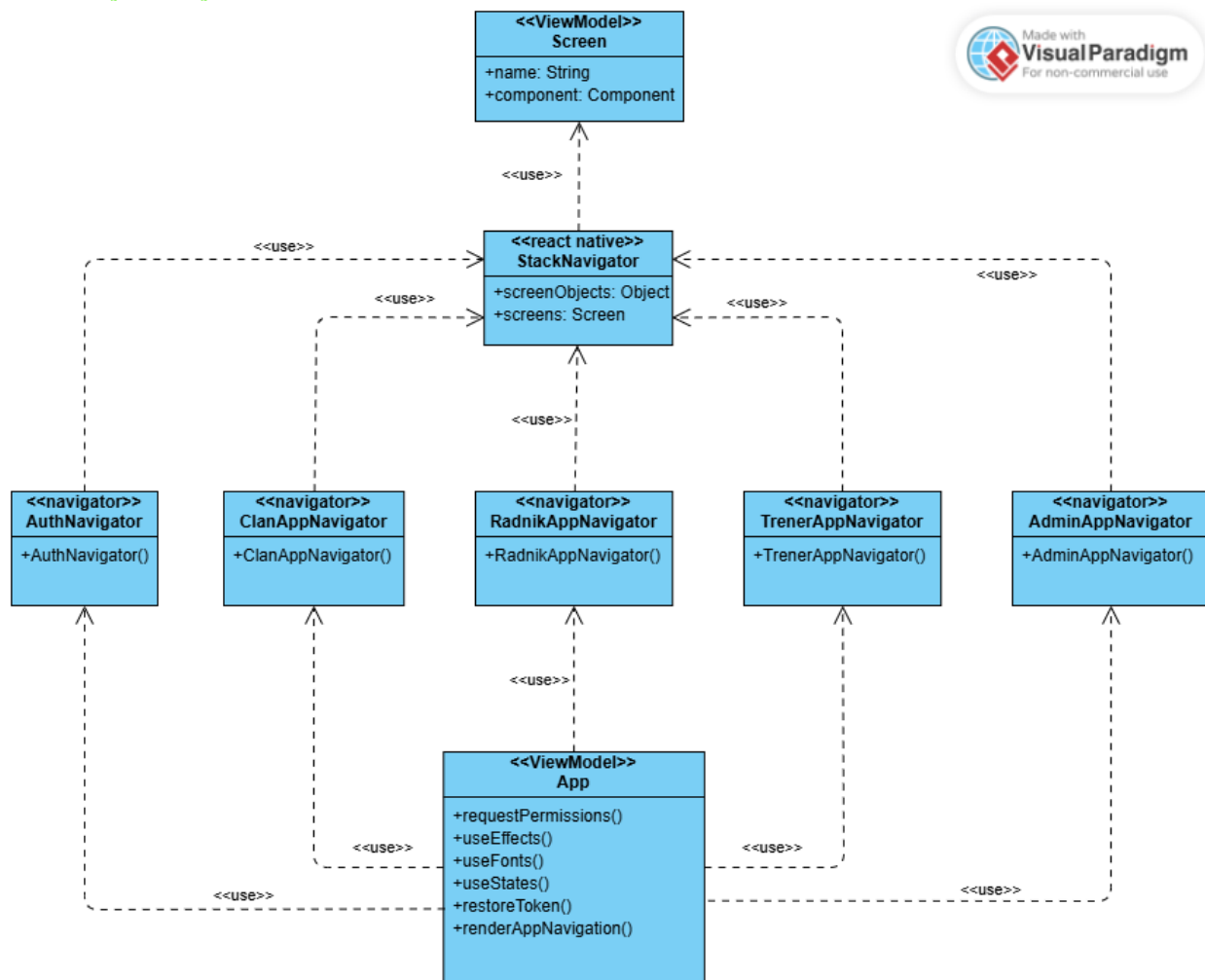
Backend dio predstavlja komunikaciju na serverskoj strani sa bazom podataka.

Tabela zaduženja		
Član Tima	Zaduženje	
Amelija Trifunović	3.2.	4.4.
Bogdana Marić	3.4.	4.2.
Milica Bobar	3.3.	4.3.
Jelena Matijaš	3.1.	4.5.
Jana Ivanković	3.5.	4.1.

```
classDiagram
    class Problem {
        +opis: String
        +status: StatusProblema
        +dohvati(PrijavljeneProbleme())
    }
    class StatusProblema {
        +NEOBRAĐEN
        +OBRAĐEN
        +U_OBRADI
    }
    class Nalog {
        +id: String
        +ime: String
        +email: String
        +lozinka: String
        +ažuriraj(Podatke())
        +odjavi()
    }
    class Obavještenje {
        +sadržaj: String
        +datum: Date
        +dohvati(Obavještenja())
    }
    class Članarina {
        +qKod: String
        +trajanje: DateTime
        +cijena: double
    }
    class Trenner {
    }
    class Zaposleni {
    }
    class Radnik {
    }
    class Administrator {
    }
    class Clan {
        +dohvati(StatusČlanarina())
        +ažuriraj(Članarina())
    }
    class Termin {
        +vrijeme: DateTime
        +trajanje: Integer
        +ažuriraj(Termin())
        +briši(Termin())
    }
    class Recenzija {
        +ocjena: Integer
        +komentar: String
    }
    class Objava {
        +sadržaj: String
        +datum: Date
        +brišanje(Objave())
    }
    class Grupa {
        +maksimalnoČlanova: Integer
        +ažuriraj(Grupe())
    }
    class PlanTreninga {
        +ime: String
        +opis: String
        +uklanjanje(PanaTreninga())
        +ažuriraj(Pana())
    }
    class ZahjevZaPracenje {
        +uklanjanje(ZahjevZaPracenja())
    }
    Problem "1" -- "*" Nalog : +prijava
    Problem "1" -- "*" Problem : +obradivanje
    Problem "1" .. "*" StatusProblema : +use
    Nalog "1" -- "*" Obavještenje : +dohvati
    Nalog "1" -- "*" Clanarina : +ma
    Nalog "1" -- "*" Clan : +starije
    Trenner "1" -- "*" Nalog : +berira
    Trenner "1" -- "*" Clan : +odabrava
    Trenner "1" -- "*" Termin : +kreira
    Trenner "1" -- "*" Recenzija : +ima
    Trenner "1" -- "*" Objava : +kreira
    Trenner "1" -- "*" Grupa : +ima
    Trenner "1" -- "*" PlanTreninga : +kreira
    Zaposleni "1" -- "*" Radnik : +sodaje
    Administrator "1" -- "*" Problem : +obradivanje
    Administrator "1" -- "*" Problem : +sodaje
    Clan "1" -- "*" Clanarina : +ma
    Clan "1" -- "*" Recenzija : +kreira
    Clan "1" -- "*" Objava : +čuva
    Clan "1" -- "*" PlanTreninga : +kreira
    Clan "1" -- "*" ZahjevZaPracenje : +starije
    Clan "1" -- "*" ZahjevZaPracenje : +uklanjanje
    Termin "1" -- "*" Recenzija : +ima
    Termin "1" -- "*" Objava : +ima
    Termin "1" -- "*" Grupa : +ima
    Termin "1" -- "*" PlanTreninga : +ima
    Recenzija "1" -- "*" Objava : +kreira
    Recenzija "1" -- "*" PlanTreninga : +ima
    Objava "1" -- "*" PlanTreninga : +ima
    Objava "1" -- "*" ZahjevZaPracenje : +kreira
    Grupa "1" -- "*" PlanTreninga : +ima
    Grupa "1" -- "*" ZahjevZaPracenje : +kreira
    PlanTreninga "1" -- "*" ZahjevZaPracenje : +kreira
    ZahjevZaPracenje "1" -- "*" ZahjevZaPracenje : +uklanjanje
```

3. Frontend

3.1. Inicijalizacija sistema



Dati dijagram klasa prikazuje arhitekturu i tok inicijalizacije klijentskog dijela aplikacije pumpd, koja je razvijena u React Native tehnologiji primjenom MVVM arhitekturnog obrasca. Dijagram ilustruje način na koji se pri pokretanju aplikacije vrši priprema resursa, verifikacija autentičnosti korisnika putem JWT tokena, te dinamičko usmjeravanje na odgovarajući navigacioni tok u zavisnosti od dodijeljene korisničke uloge (Role-Based Access Control).

Centralna ulazna tačka sistema definisana je klasom App, koja se nalazi u ViewModel sloju i upravlja početnim stanjem i konfiguracijom aplikacije kroz sljedeće metode:

- `requestPermissions()` – za zahtijevanje i provjeru sistemskih dozvola na mobilnom uređaju (npr. notifikacije, pristup skladištu),
- `useEffects()` – za pokretanje sporednih efekata i asinhronih operacija pri inicijalizaciji,
- `useFonts()` – za učitavanje i pripremu prilagođenih fontova aplikacije,
- `useStates(user: User)` – za upravljanje centralnim stanjem korisnika i aplikativnih podataka,

- `restoreToken()` – za provjeru i obnavljanje JWT autentifikacionog tokena iz lokalnog skladišta radi utvrđivanja statusa prijave,
- `renderAppNavigator(accountType: String)` – za dinamičko prikazivanje i pokretanje odgovarajućeg navigatora na osnovu uloge prijavljenog korisnika.

Na osnovu uloge korisnika dobijene prilikom inicijalizacije, klasa `App` aktivira jedan od specijalizovanih navigatora koji nose stereotip «navigator»:

- `AuthNavigator` – upravlja navigacionim tokom za neprijavljene korisnike (ekrani za prijavu i registraciju),
- `ClanAppNavigator` – definisano okruženje i tok navigacije namijenjen ulozi Člana,
- `TrenerAppNavigator` – navigacioni tok sa funkcionalnostima prilagođenim ulozi Trenera,
- `RadnikAppNavigator` – navigacioni tok sa opcijama namijenjenim ulozi Radnika,
- `AdminAppNavigator` – navigacioni tok sa pristupom administrativnim funkcionalnostima sistema.

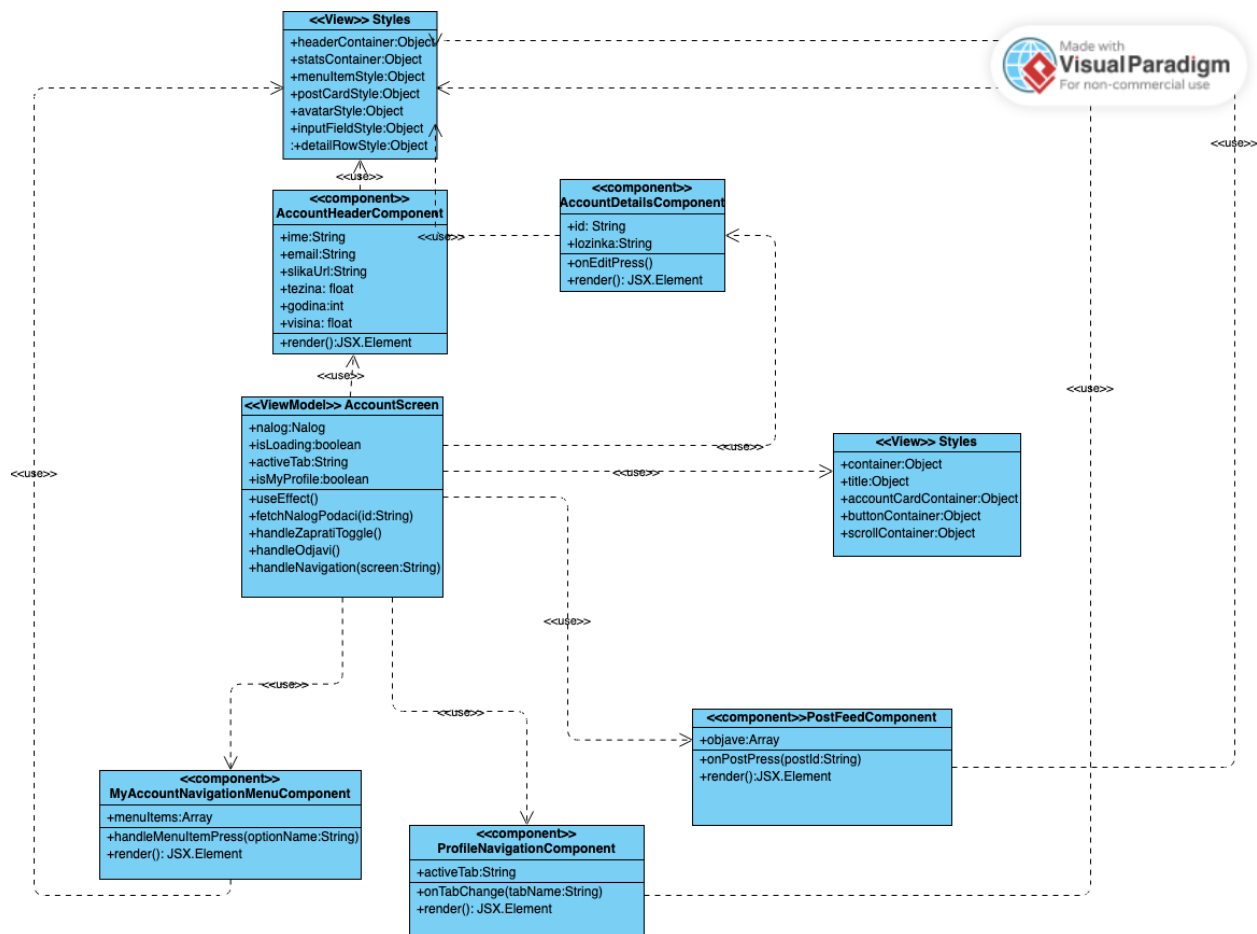
Svi ulogom specifični navigatori oslanjaju se na `StackNavigator` (sa stereotipom «react native»), koji upravlja stogom ekrana i sadrži attribute `screenObjects: Object` i `screens: Screen`. Na samom vrhu hijerarhije nalazi se klasa `Screen` (ViewModel sloj) sa atributima `name: String` i `component: Component`, koja reprezentuje pojedinačne GUI ekrane u aplikaciji.

Zavisnosti između slojeva definisane su usmjerenim «use» relacijama: klasa `App` koristi odgovarajući `AppNavigator`, svaki navigator koristi `StackNavigator` za kretanje kroz aplikaciju, dok `StackNavigator` upravlja i prikazuje konkretne `Screen` komponente.

Ovakav arhitektonski pristup inicijalizaciji omogućava:

- siguran i brz proces pokretanja aplikacije uz provjeru autorizacije (JWT),
- potpunu izolaciju funkcionalnih tokova zavisno od uloge korisnika (RBAC),
- modularnost i jednostavna proširenja dodavanjem novih ekrana ili uloga bez narušavanja postojeće strukture.

3.2. Pregled naloga



U okviru sistema pumpd, korisnički interfejs za pregled i upravljanje korisničkim nalogom organizovan je kroz jasno strukturirane ekrane i komponente korišćenjem React Native tehnologije. Klasa **AccountScreen** implementirana je kao glavni ekran (screen klasa) i predstavlja **ViewModel** sloj koji upravlja prikazom i logikom naloga korisnika. Funkcionalnost i izgled ovog ekrana zasnivaju se na ponovnom korišćenju specijalizovanih reupotrebljivih komponenti, čime se postiže visoka modularnost i fleksibilnost prilikom obrade različitih tipova profila (sopstveni profil ili profil trenera).

AccountScreen upravlja logikom i prikazom pomoću sledećih React komponenti:

- **AccountHeaderComponent** - zadužena za prikaz osnovnih personalnih i fizičkih parametara korisnika, kao što su ime, email, slika profila (*slikaUrl*), težina, godine i visina.
- **AccountDetailsComponent** -komponenta koja se koristi za prikaz i izmjenu osjetljivih detalja naloga, poput lozinke, uz integrisanu metodu *onEditPress* za pokretanje izmjena.
- **MyAccountNavigationMenuComponent** -specijalizovani navigacioni meni koji se prikazuje na sopstvenom profilu korisnika (kada je *isMyProfile*: boolean postavljeno na true), pružajući opcije poput pregleda sačuvanih objava, prijave problema, provjere članarine i odjave.

- ProfileNavigationComponent -komponenta zadužena za tab navigaciju (objave, recenzije, grupe) prilikom pregleda profila.
- PostFeedComponent -komponenta koja služi za prikaz objava trenera, sa podrškom za obradu pritiska na pojedinačnu objavu putem metode onPostPress.

Ove komponente zajedno sa AccountScreen klasom sačinjavaju ViewModel sloj, jer pored samog prikaza sadrže i funkcionalnosti neophodne za ponašanje interfejsa, kao što su obrada događaja (klikovi, navigacija, prebacivanje tabova) i upravljanje stanjem (isLoading, activeTab, isMyProfile). Klasa AccountScreen uključuje ključne metode kao što su useEffect i fetchNalogPodaci za dohvaćanje podataka o nalogu, handleZapratiToggle za praćenje trenera, handleNavigation za usmjeravanje korisnika, te handleOdjavi za odjavu sa sistema.

Prikaz korisničkog interfejsa dodatno je podržan klasama Styles, koje predstavljaju View sloj. Dizajn je podijeljen na dvije Styles klase radi lakšeg održavanja:

- Glavna Styles klasa ekrana -definiše globalni raspored i kontejner samog ekrana kroz stilove container, title, accountCardContainer, buttonContainer i scrollContainer.
- Komponentna Styles klasa -definiše izgled unutrašnjih elemenata i podržava pojedinačne komponente kroz stilove kao što su headerContainer, avatarStyle, statsContainer, menuItemStyle, postCardStyle, inputFieldStyle i detailRowStyle.

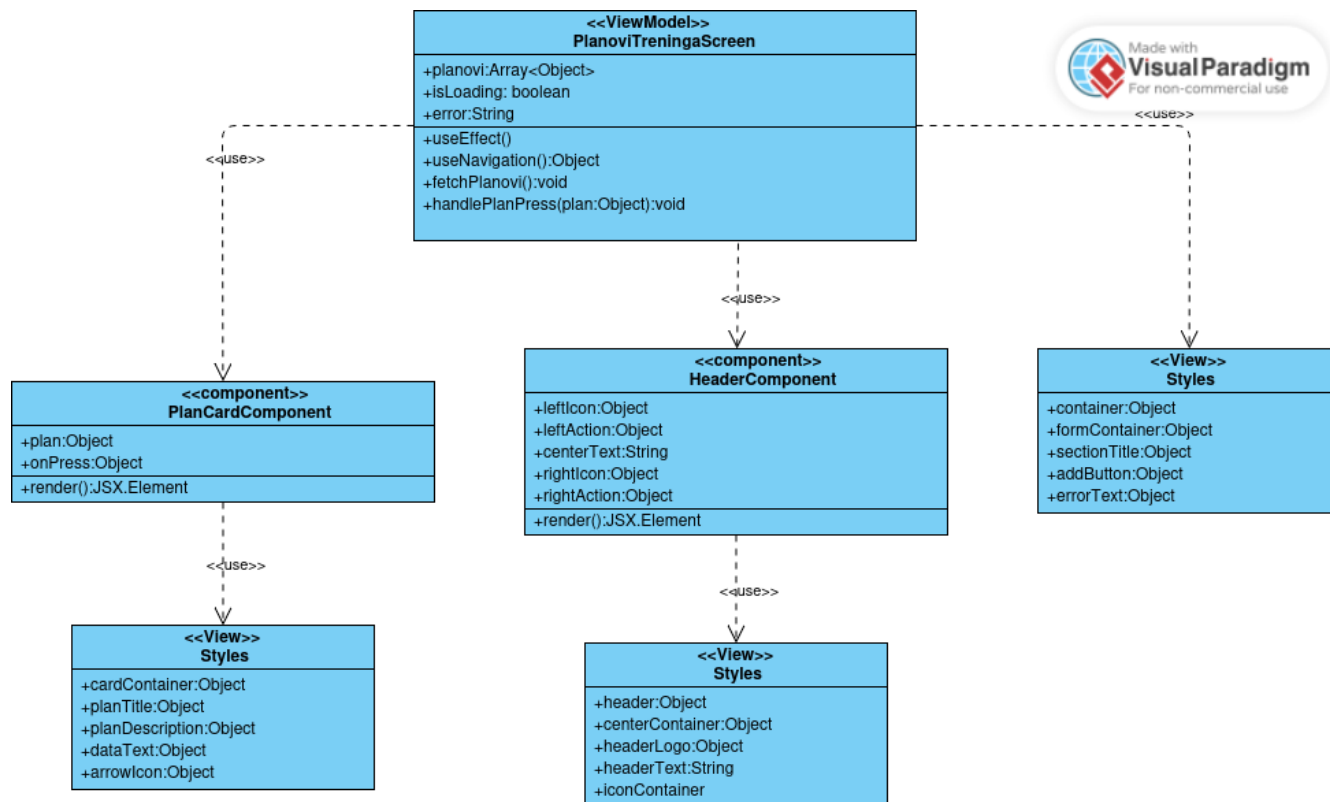
Korisnički tok omogućava da se zavisno od konteksta (da li korisnik pristupa svom profilu ili profilu trenera) dinamički prikazuju odgovarajuće komponente i stilovi.

Ovakav arhitektonski pristup omogućava:

- ponovnu upotrebu UI komponenti,
- lako prilagođavanje prikaza zavisno od uloge korisnika (isMyProfile),
- jasno razdvajanje logike prikaza, ponašanja i stilova,

što čini sistem fleksibilnim, skalabilnim i jednostavnim za dalje proširenje.

3.3. Pregled planova treninga

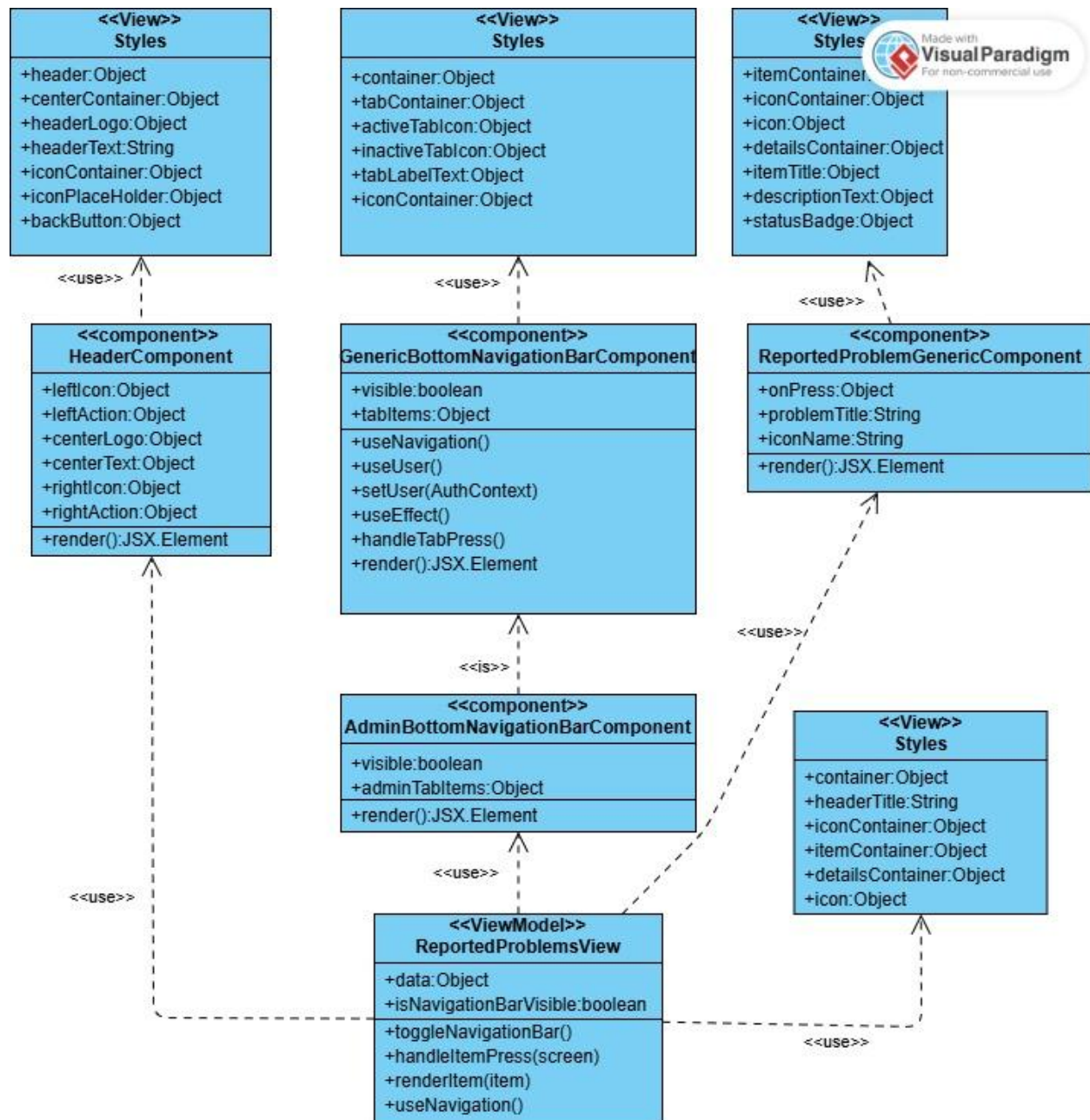


Dijagram prikazuje ekran za pregled liste planova treninga. Glavna logika nalazi se u klasi `PlanoviTreningaScreen`, koja predstavlja `ViewModel` i upravlja dohvaćanjem i prikazivanjem podataka o planovima. Prilikom otvaranja ekrana, metoda `useEffect()` poziva `fetchPlanovi()` koja dohvata listu planova sa servera, dok se stanje učitavanja i eventualne greške prate kroz atribute `isLoading` i `error`.

Svaki plan u listi prikazuje se putem komponente `PlanCardComponent`, koja sadrži osnovne podatke o planu (naziv, opis, datum) i omogućava korisniku da klikom otvori detalje odabranog plana putem metode `handlePlanPress(plan)`. Zaglavlje ekrana prikazano je kroz komponentu `HeaderComponent`, koja se koristi na više ekrana aplikacije radi konzistentnog izgleda.

Stilizacija je odvojena od logike kroz zasebne `Styles` klase za svaku komponentu, čime je poštovan princip razdvajanja prikaza (`View`) od ponašanja (`ViewModel`), u skladu sa arhitekturom opisanom u uvodu dokumenta.

3.4 Pregled prijavljenih problema



Dati dijagram klasa odnosi se na frontend klijentskog podsistema aplikacije *pumpd*, koji administratoru omogućava pregled i upravljanje prijavljenim problemima.

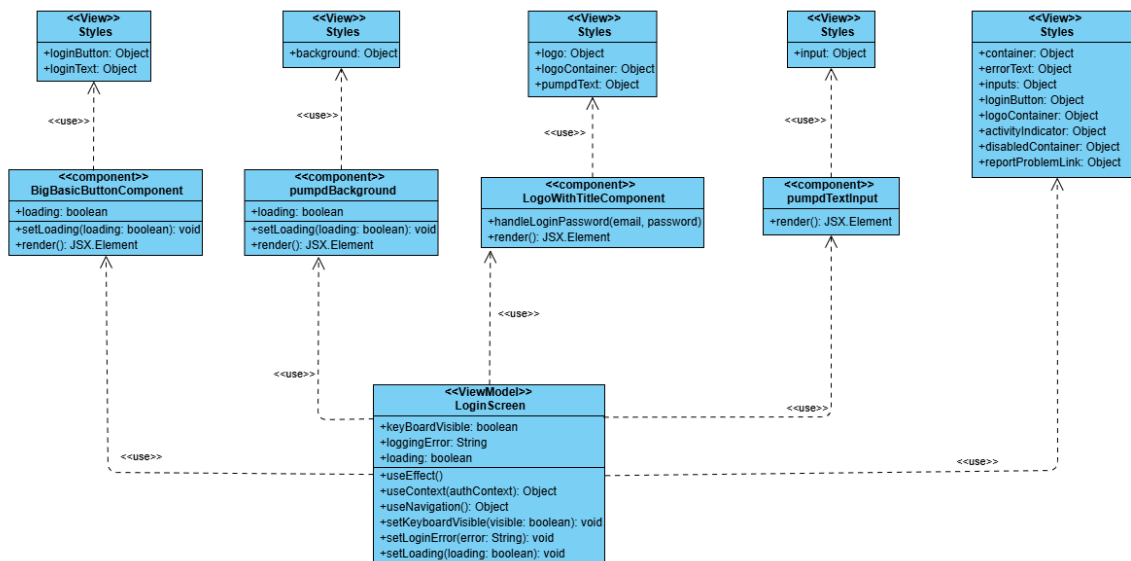
Dijagram obuhvata sljedeće cjeline i komponente:

- AdminBottomNavigationBarComponent predstavlja specijalizaciju generičke komponente GenericBottomNavigationBarComponent. Koristi se za prikaz primarne donje navigacione trake (Bottom Tab Bar), obezbjeđujući administratoru brzu navigaciju između ekrana.

- ReportedProblemsView predstavlja ViewModel koji upravlja prikazom liste prijavljenih problema i kontrolom navigacione trake. Sadrži metode za navigaciju, dinamiku prikaza i obradu pojedinačnih prijava.
- ReportedProblemGenericComponent je višekratna komponenta zadužena za vizuelni prikaz pojedinačnih kartica prijavljenih problema (sa ikonom ključa i nazivom), uz podršku za interakciju putem događaja onPress.
- HeaderComponent definiše gornje zaglavlje ekrana sa elementima poput dugmeta za nazad (backButton), centralnog logotipa i ikona za brze akcije (logout/notifikacije/podešavanja).

Sve komponente su povezane sa odgovarajućim Styles klasama («View») koje izdvajaju vizuelno oblikovanje iz same logike prikaza. Na taj način jasno su razdvojeni izgled, logika i navigacija, što obezbeđuje visoku skalabilnost i lakše održavanje korisničkog interfejsa mobilne aplikacije.

3.5 Prijava na sistem



U okviru aplikacije pumpd, korisnički interfejs za autentifikaciju korisnika organizovan je kroz jasno strukturirane ekrane i komponente korišćenjem React Native tehnologije. Klasa LoginScreen implementirana je kao ekran (screen klasa) koji predstavlja samostalnu funkcionalnu cjelinu za prijavu korisnika na aplikaciju. Njena funkcionalnost i izgled zasnivaju se na ponovnom korišćenju specijalizovanih komponenti, čime se postiže modularnost i efikasnost prilikom razvoja korisničkog interfejsa.

Konkretno, LoginScreen je sastavljen od sljedećih React komponenti:

- BigBasicButtonComponent – služi za prikaz glavnog dugmeta za prijavu uz podršku za vizuelnu indikaciju učitavanja (loading).
- pumpdBackground – komponenta zadužena za prikaz pozadinskog dizajna i vizuelnog okvira specifičnog za aplikaciju pumpd.
- pumpdTextInput – prilagođena ulazna komponenta za unos korisničkih podataka (email i password).
- LogoWithTitleComponent – prikazuje prepoznatljivi logo aplikacije u obliku bučice sa natpisom "pumpd", uz integrisanu logiku za obradu prijave putem metode handleLoginPassword.

Ove komponente zajedno čine ViewModel sloj, jer osim prikaza sadrže i funkcionalnosti neophodne za ponašanje ekrana – kao što su obrade događaja (klikova, unosa, grešaka) i promjene stanja (loading, keyboardVisible, loggingError). Na primjer, klasa LoginScreen uključuje metode za upravljanje stanjem autentifikacije, navigacijom i prikazom tastature.

Prikaz korisničkog interfejsa dodatno je podržan klasama Styles, koje predstavljaju View sloj. Svaka komponenta koristi zaseban Styles objekat koji definiše izgled elemenata kroz stilove poput

container, inputs, loginButton, logoContainer, reportProblemLink, activityIndicator i drugih. Na ovaj način omogućava se fleksibilna vizuelna prezentacija svakog dijela interfejsa, uz zadržavanje funkcionalne logike odvojene od prikaza.

Korisnički tok započinje prikazom LoginScreen kao inicijalnog ekrana za autentifikaciju. Korisnik unosi svoje pristupne podatke (e-mail i lozinku) i potvrđuje prijavu na sistem, a na raspolaganju mu je i opcija za prijavu problema (reportProblemLink) ukoliko naiđe na poteškoće pri pristupu. Time se obezbjeđuje jednostavna, direktna i intuitivna navigacija unutar autentifikacijskog dijela aplikacije pumpd.

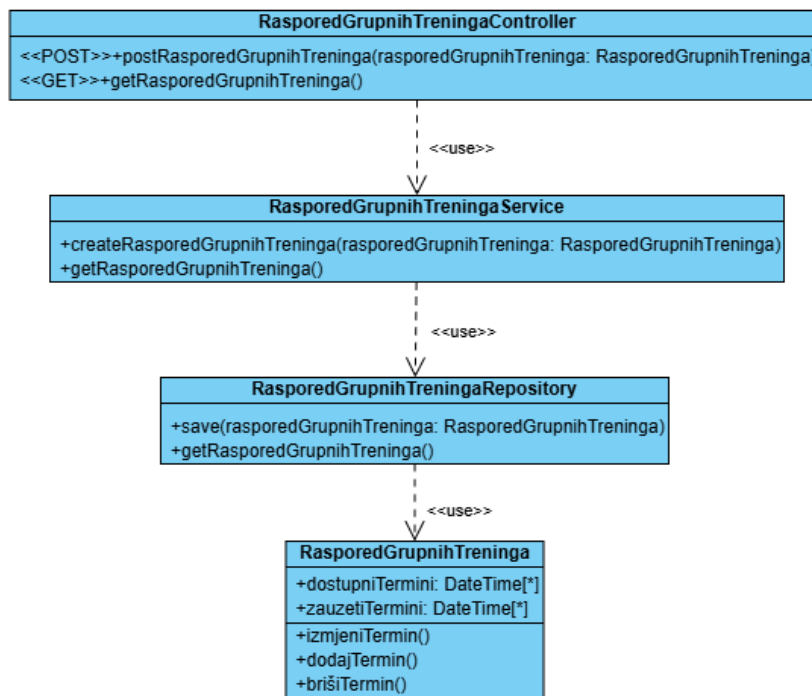
Ovakav arhitektonski pristup omogućava:

- ponovnu upotrebu komponenti,
- bolju testabilnost i održavanje,
- jasno razdvajanje logike i prikaza,

što čini sistem fleksibilnim i spremnim za dalje proširenje.

4. Backend

4.1. Raspored grupnih treninga



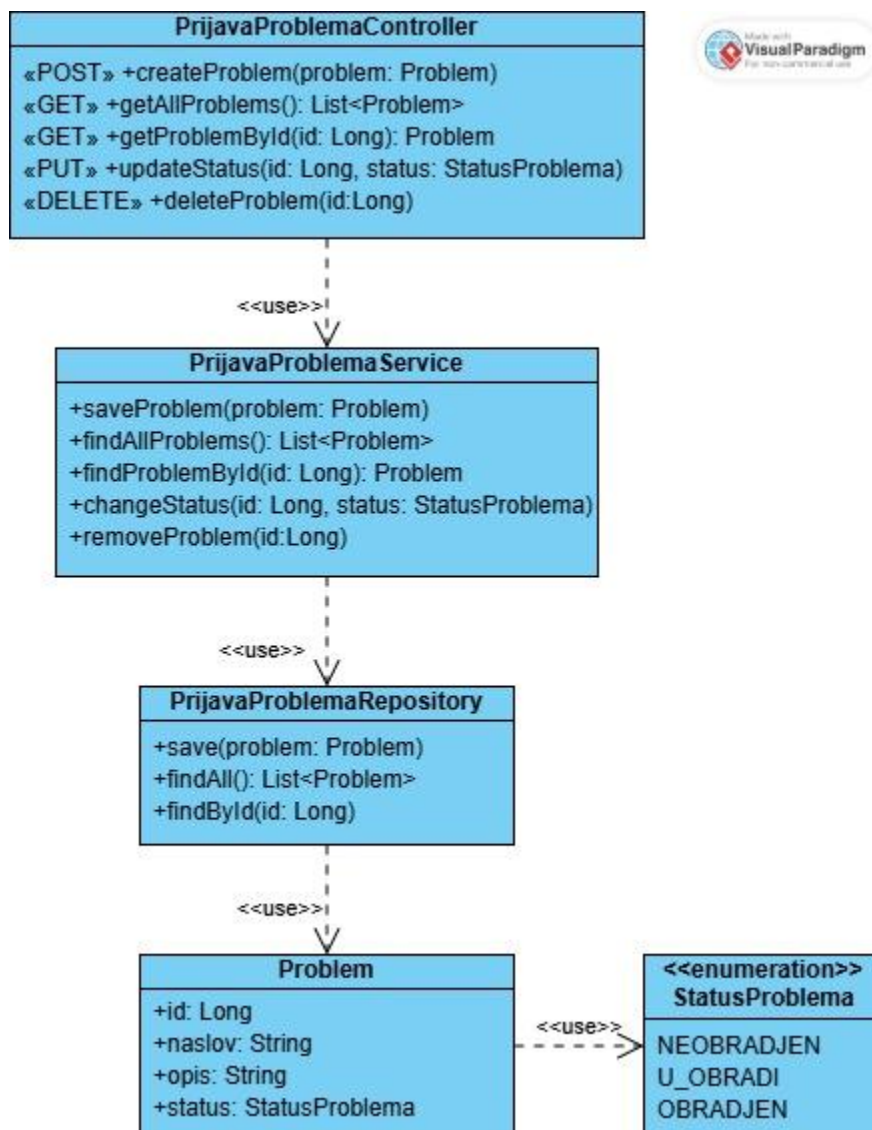
Prikazani dijagram klasa ilustruje backend strukturu modula za upravljanje rasporedom grupnih treninga, koji omogućava komunikaciju servera sa bazom podataka radi kreiranja i pregleda termina. Modul je izrađen po principima Spring Boot i JPA arhitekture kroz slojeviti pristup: Controller – Service – Repository. Za domensku klasu RasporedGrupnihTreninga, koja sadrži atribut dostupniTermini i zauzetiTermini te metode za njihovu izmjenu, definisane su tri komponente:

- RasporedGrupnihTreningaController obrađuje HTTP zahteve i izlaže REST endpointe:
 - «POST» postRasporedGrupnihTreninga(rasporedGrupnihTreninga: RasporedGrupnihTreninga) – za evidenciju novog rasporeda,
 - «GET» getRasporedGrupnihTreninga() - za preuzimanje važećeg rasporeda.
- RasporedGrupnihTreningaService sadrži poslovnu logiku i posreduje između kontrolera i repozitorijuma putem metoda createRasporedGrupnihTreninga i getRasporedGrupnihTreninga.
- RasporedGrupnihTreningaRepository komunicira direktno sa bazom podataka radi trajne pohrane, koristeći operacije save i getRasporedGrupnihTreninga.

Komunikacija između slojeva odvija se sekvencijalno putem «use» relacija, gdje kontroler poziva servis, a servis repozitorijum radi manipulacije nad entitetom RasporedGrupnihTreninga.

Ovaj model jasno oslikava odvojenost slojeva i podjelu odgovornosti, što omogućava lakše održavanje i proširivost sistema u aplikaciji pumpd.

4.2. Prijava problema



Prikazani dijagram klasa ilustruje backend strukturu modula za upravljanje prijavljenim problemima u okviru *pumpd* sistema, koji omogućava sigurnu komunikaciju između serverskog dijela i baze podataka. Dijagram je projektovan po principima višeslojne (Layered) Spring Boot arhitekture u kombinaciji sa JPA (Java Persistence API) pristupom.

Struktura se sastoji od sljedećih ključnih slojeva i klasa:

- PrijavaProblemaController predstavlja REST kontroler koji prima HTTP zahtjeve sa frontenda i izlaže endpointe za kreiranje novih prijava (POST), dohvaćanje svih prijava ili pojedinačnog problema prema ID-ju (GET), za ažuriranje statusa obrade od strane administratora (PUT) kao i brisanje prijava problema iz Sistema (DELETE).
- PrijavaProblemaService sadrži poslovnu logiku aplikacije. Implementira metode za čuvanje prijava, pretragu svih problema ili problema po identifikatoru, te logiku za izmjenu statusa prijavljenog problema i uklanjanje prijavljenog problem na osnovu identifikatora.

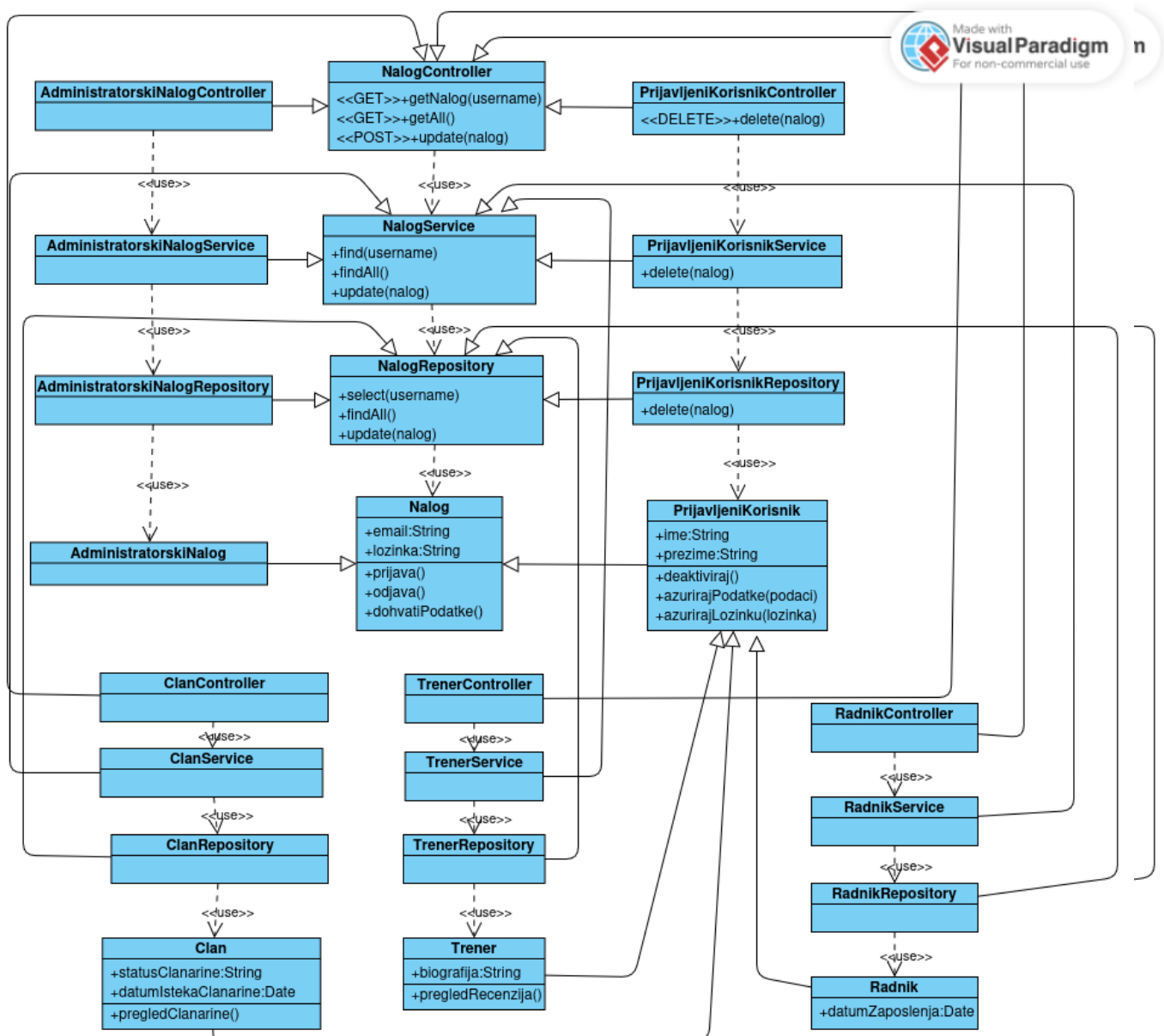
- `PrijavaProblemaRepository` predstavlja sloj pristupa podacima koji komunicira direktno sa bazom podataka preko standardnih JPA metoda poput `save`, `findAll` i `findById`.

`Problem` je domenska entitetska klasa (Model) koja mapira podatke iz baze i sadrži attribute `id`, `naslov`, `opis` i `status`.

`StatusProblema` je enumeracija («enumeration») koja definiše stanja u kojima se prijava može nalaziti: `NEOBRAĐEN`, `U_OBRADI` i `OBRAĐEN`.

Dijagram jasno prikazuje odvojenost odgovornosti kroz trostruki sloj (Controller – Service – Repository), čime je obezbijeđena visoka modularnost, laka održivost i jednostavna proširivost backend podsistema.

4.3. Administratorski i korisnički nalog



Dati dijagram klasa predstavlja komunikaciju servera sa bazom podataka u okviru Spring Boot aplikacije koja koristi JPA za rad sa različitim vrstama korisničkih naloga. Svaka domena je implementirana po obrascu Controller–Service–Repository, čime je ostvaren jasan slojeviti pristup: Controller upravlja HTTP zahtjevima, Service sadrži poslovnu logiku, a Repository služi za pristup podacima.

Centralna klasa je Nalog, koja sadrži osnovne atribute (email, lozinka) i metode za prijavu, odjavu i dohvat podataka (prijava(), odjava(), dohvatiPodatke()). Na osnovu nje izvedene su dvije grane naloga:

- AdministratorskiNalog, koji direktno nasljeđuje Nalog i predstavlja poseban tip naloga sa sopstvenim slojem AdministratorskiNalogController–AdministratorskiNalogService–AdministratorskiNalogRepository.
- PrijavljeniKorisnik, koji takođe nasljeđuje Nalog i predstavlja zajedničku apstrakciju za sve registrovane korisnike sistema. Ova klasa dodaje attribute ime i prezime, kao i metode deaktiviraj(), azurirajPodatke(podaci) i azurirajLozinku(lozinka), koje su zajedničke svim registrovanim korisnicima.

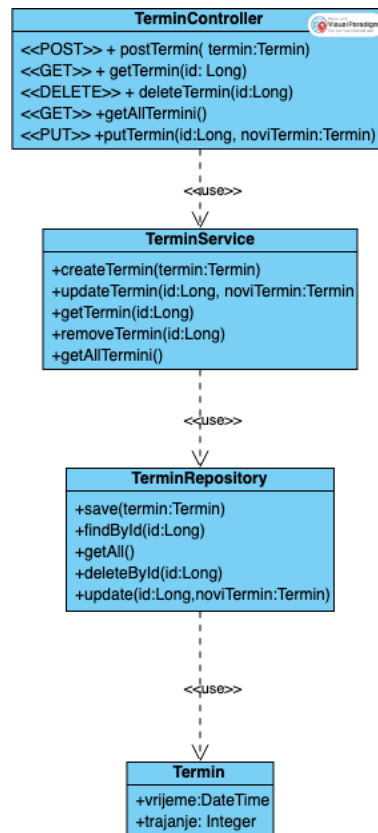
Iz klase PrijavljeniKorisnik dalje su izvedene tri specijalizovane klase, svaka sa sopstvenim Controller–Service–Repository slojem:

- Clan – sa atributima statusClanarine i datumIstekaClanarine, te metodom pregledClanarine().
- Trener – sa atributom biografija i metodom pregledRecenzija().
- Radnik – sa atributom datumZaposlenja.

Zajedničke operacije nad nalogima (dohvatanje po korisničkom imenu, dohvatanje svih naloga i ažuriranje) definisane su u klasama NalogController, NalogService i NalogRepository, koje koriste i AdministratorskiNalogController i svi kontroleri izvedeni iz PrijavljeniKorisnik grane. Dodatno, PrijavljeniKorisnikController, PrijavljeniKorisnikService i PrijavljeniKorisnikRepository uvode funkcionalnost brisanja naloga (delete(nalog)), koju dijele i ponovo koriste ClanController, TrenerController i RadnikController.

Ovakav dizajn omogućava jasno razdvajanje zajedničke logike (u klasama Nalog i PrijavljeniKorisnik) od logike specifične za svaki tip korisnika, uz minimalno dupliranje koda kroz ponovnu upotrebu zajedničkih slojeva.

4.4. Upravljanje terminima



Dati dijagram klasa prikazuje backend strukturu modula za upravljanje terminima, koji omogućava komunikaciju servera sa bazom podataka radi kreiranja, pregleda, ažuriranja i brisanja informacija o terminima. Dijagram je izrađen prema principima Spring Boot arhitekture u kombinaciji sa JPA (Java Persistence API) pristupom, pri čemu je svaka domena implementirana kroz jasan slojeviti pristup: Controller – Service – Repository.

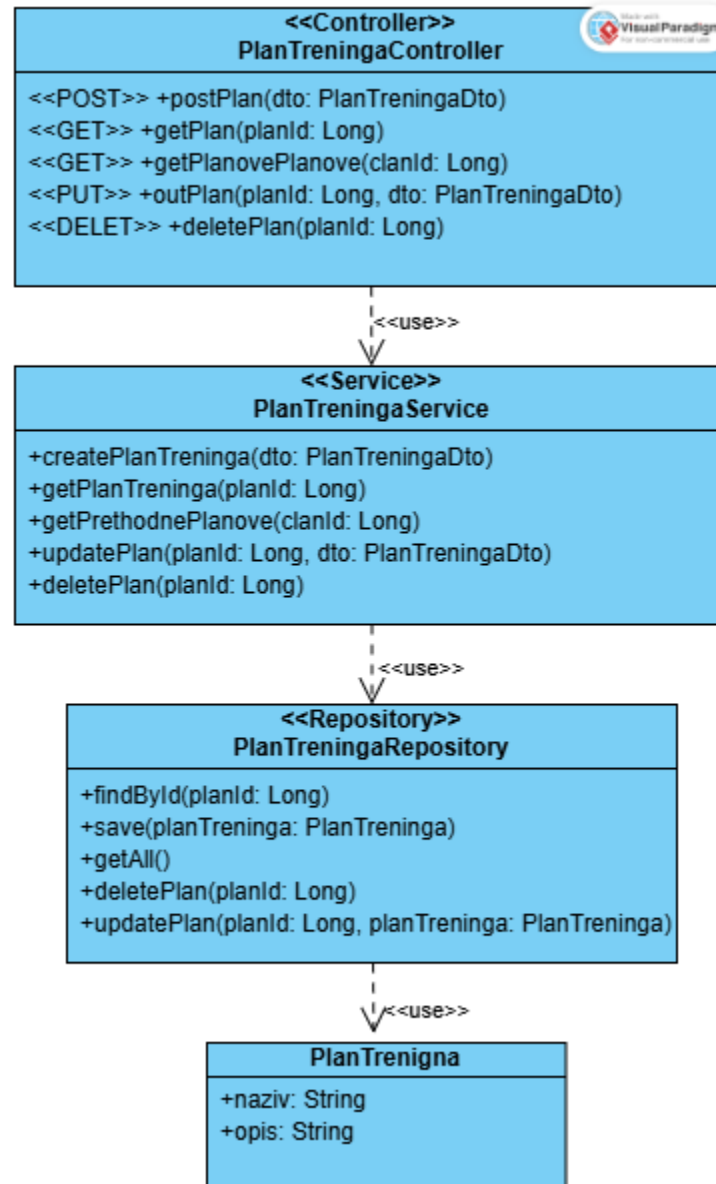
Za domensku klasu Termin, koja sadrži osnovne attribute vrijeme: DateTime i trajanje: Integer, definisane su tri odgovarajuće klase koje čine tipičan trostruki sloj arhitekture:

- TerminController upravlja HTTP zahtjevima i izlaže REST endpointe za obradu termina:
 - «POST» postTermin(termin: Termin) - za kreiranje novog termina,
 - «GET» getTermin(id: Long) - za dohvaćanje pojedinačnog termina na osnovu njegovog ID-ja,
 - «GET» getAllTermini() - za dohvaćanje liste svih termina,
 - «PUT» putTermin(id: Long, noviTermin: Termin) – za ažuriranje postojećeg termina,
 - «DELETE» deleteTermin(id: Long) – za brisanje termina.

- TerminService sadrži poslovnu logiku i posreduje između kontrolera i repozitorijuma kroz metode createTermin, updateTermin, getTermin, removeTermin i getAllTermini.
- TerminRepository komunicira direktno sa bazom podataka radi perzistencije podataka, koristeći standardne JPA operacije kao što su save, findById, getAll, deleteById i update.

Komunikacija između slojeva odvija se hijerarhijski putem «use» relacija, gdje TerminController koristi TerminService, koji potom poziva TerminRepository za rad nad samim entitetom Termin. Ukratko, ovaj dijagram jasno ilustruje odvojenost slojeva u arhitekturi, modularnost i princip odvajanja odgovornosti, što omogućava laku održivost i proširivost sistema u skladu sa potrebama.

4.5 Upravljanje planovima treninga



Dati dijagram klasa prikazuje backend strukturu modula za upravljanje planovima treninga, koji omogućava komunikaciju servera sa bazom podataka radi kreiranja, pregleda, ažuriranja i brisanja informacija o planovima treninga. Dijagram je izrađen prema principima Spring Boot arhitekture u kombinaciji sa JPA (Java Persistence API) pristupom, pri čemu je svaka domena implementirana kroz jasan slojeviti pristup: Controller – Service – Repository.

Za domensku klasu PlanTreninga, koja sadrži osnovne attribute naziv: String i opis: String, definisane su tri odgovarajuće klase koje čine tipičan trostruki sloj arhitekture:

- PlanTreningaController upravlja HTTP zahtjevima i izlaže REST endpointe za obradu planova treninga:
 - «POST» postPlanTreninga(planTreninga: PlanTreninga) – za kreiranje novog plana treninga,
 - «GET» getPlanTreninga(id: Long) – za dohvaćanje pojedinačnog plana treninga na osnovu njegovog ID-ja,
 - «GET» getAllPlanoviTreninga() – za dohvaćanje liste svih planova treninga,
 - «PUT» putPlanTreninga(id: Long, noviPlanTreninga: PlanTreninga) – za ažuriranje postojećeg plana treninga,
 - «DELETE» deletePlanTreninga(id: Long) – za brisanje plana treninga.
- PlanTreningaService sadrži poslovnu logiku i posreduje između kontrolera i repozitorijuma kroz metode createPlanTreninga, updatePlanTreninga, getPlanTreninga, removePlanTreninga i getAllPlanoviTreninga.
- PlanTreningaRepository komunicira direktno sa bazom podataka radi perzistencije podataka, koristeći standardne JPA operacije kao što su save, findById, getAll, deleteById i update.

Komunikacija između slojeva odvija se hijerarhijski putem «use» relacija, gdje PlanTreningaController koristi PlanTreningaService, koji potom poziva PlanTreningaRepository za rad nad samim entitetom PlanTreninga.

Ukratko, ovaj dijagram jasno ilustruje odvojenost slojeva u arhitekturi, modularnost i princip odvajanja odgovornosti, što omogućava laku održivost i proširivost sistema u skladu sa potrebama.